Arjun T
AMB.2024

Python – Advanced Activity Handouts (Project & Problem-Solving Focus)

Activity A1 – From Idea to Project Specification

Title: Define Your Mini-Project Clearly

Goal
Turn a rough idea into a precise, buildable project description.

Instructions
Pick one idea (examples: simple hospital appointment system, dataset analyzer, quiz app, log analyzer, file organizer).

Part A – One-page project brief

1. Project title:

Hospital appointment system

2. Problem you want to solve (2–3 short sentences):

Patients often forget clinic appointments and hospital staff need an easy way to manage bookings - need a tool to book, view appointments

3. Target users (who will actually use this?):

Hospital receptionists, Patient

4. Inputs (what data / actions will the program receive?):
   - Patient name, Appointments and date and time
   - Phone number and Commands
5. Outputs (what will the user see / get?):
   - Confirmation message, list of all appointments for today
   - Message when an appointment is deleted
6. Constraints or assumptions (keep it realistic):
   - Emergency walk-in
   - incase the Patient doesn't give up

Part B – Core features vs "nice-to-have"

1. List minimum features needed for a working version (MVP):
   - Add appointments
   - View all appointments
   - Delete an appointment
2. List optional features (only if time allows):
   - Sent sms / email
   - Search appointments by date
3. Final question:
   - If you had only 1 week, which ONE feature would you absolutely keep? Why?

I would keep the "add new appointment" feature Because that is the function of project

Activity A2 – System Decomposition (Modules & Responsibilities)

Title: Break Project into Modules

Goal
Learn to split a project into logical modules and functions.

Instructions
Use your project from Activity A1.

Part A – High-level modules

1. List 3–6 modules (e.g., data_input, processing, reporting, ui):
   - Module 1: Patient Registration
   - Module 2: Appointment scheduling
   - Module 3: Doctor availability
   - Module 4: Notification System
2. For each module, write its main responsibility in one sentence:
   - Module 1 does: Capture Patient details an store them
   - Module 2 does: manage booking and rescheduling
   - Module 3 does: maintain and Provide doctor time slot
   - Module 4 does: Send reminder and Confirmation to Patient an doctor

Part B – Functions inside modules

For one key module (choose the most important):

1. Module name: Appointment scheduling
2. Functions planned:
   - Function 1 name: Check slot availability
     - Input(s): doctor ID, desired date and time
     - Output: boolean (slot free or not)
   - Function 2 name: Book appointment
     - Input(s): Patient ID, doctor ID, slot time
     - Output: appointment Confirmation ID
   - Function 3 name: Reschedule appointment
     - Input(s): existing appointment ID, new date and time
     - Output: Updated appointment Confirmation
3. Draw or briefly describe how data flows between these functions:

Activity A3 – Designing Data Structures

Title: Choosing the Right Data Structures

Goal

Practice selecting lists, dicts, sets, tuples, or custom classes.

Instructions

Consider your project's core data (e.g., patient records, quiz questions, log entries, transactions).

Part A – Describe one key entity

1. Entity name (e.g., "Appointment", "Record", "Transaction"):

   Appointment

2. For this entity, list required fields:

   - Field 1: Appt ID (type: int/str/float/...)
   - Field 2: Patient name (type: string)
   - Field 3: Dr. name (type: string)
   - Field 4: Appointment date and time (type: int)

Part B – Pick implementation

Choose one representation and justify:

1. Option A – Dictionary:

python

```
item = {
  "field1": app-id,
  "field2": Patient name
  ...
}
```

2. Option B – Class:

python

```
class Item:
  def __init__(self, field1, field2, ...):
    self.field1 = field1
    self.field2 = field2
    ...
```

3. Which option will you use, and why?

   Option B - Class is more robust, easy editing appointments

4. Write a short example of creating and using this structure:

python

```
# Example creation and simple usage
```

Arjun T

Python Foundations – Activity Handouts

Activity 0.1 – Everyday Task as an Algorithm

Title: From Daily Routine to Algorithm

Instructions to Students

1. Choose one simple daily activity (examples: making tea, preparing to go to class, booking a cab).
2. Write the steps in clear, numbered points.
3. Make sure each step is:
   - Unambiguous
   - In correct order
   - Does not skip important details

Template

1. Activity chosen: Booking a Cab
2. Write the steps:
   1. Open the app
   2. Select the "Pick up" location
   3. Select the "~~Dropping~~ Drop off" location
   4. Select the type of ride you Prefer (Premium, XL,
   5. Confirm the ride
   6. Wait for the driver to reach the location of "Pick up".
3. Check your algorithm:
   - Can a friend follow it without asking questions? Yes / No
   - If "No", refine the steps here:
     - _____

Activity 0.2 – From Steps to Pseudo-code

Title: Writing Pseudo-code with IF and REPEAT

Instructions

1. Take your steps from Activity 0.1.
2. Identify where decisions happen (IF/ELSE).
3. Identify where repetition happens (REPEAT / WHILE).
4. Rewrite your algorithm using the following keywords:
   - IF, ELSE, ELSE IF
   - REPEAT, WHILE, UNTIL

Template

1. Original activity: Booking a Cab.
2. Pseudo-code version:

text

START

Open the app and select ~~drop~~ "Pick up" and "drop off" locati

IF desired type of Cab available THEN

Confirm the ~~book~~ ride.

ELSE

Choose any other type of Cab (XL, Premium AC, Comfort)

ENDIF

Desired type of Cab is available.

REPEAT

For desired type of Cab

UNTIL Desired type of Cab get booked.

END

3. Reflection:
   - Where did you use IF/ELSE? For checking desired Cab is available
   - Where did you use REPEAT/WHILE? For repeating process until desired cab is booked

Activity 2.1 – Calculator in Steps

Title: From Word Problem to Code

Problem 1
A student has marks in 3 subjects. Find total and average.

1. Write the steps in plain language:
   1. Input the marks of 3 subjects
   2. Calculate the total marks by adding all 3 marks
   3. Calculate avg marks dividing by 3
   4. Display total and average.
2. Now convert to Python (fill in the blanks):

python

mark1 = int (input ("Enter mark 1 :"))

mark2 = int (input ("Enter mark 2 :"))

mark3 = int (input ("Enter mark 3 :"))

total = mark 1 + mark 2 + mark 3

average = total /3

print("Total =", total)

print("Average =", average)

3. Test with your own numbers and record output.

Activity 2.2 – Fix the Errors

Title: Debugging Simple Python

Instructions
Each snippet has an error.

1. Write what you think is wrong.
2. Correct the code.

Snippet 1

python

print("Hello World)

- Error: Missing quotation mark
- Corrected code: ("Hello World")

python

undefined

Snippet 2

python

Activity 3.1 – Rule-based Classification

Title: Grades with IF/ELSE

Rule

- If mark ≥ 90 : Grade A
- Else if mark ≥ 75 : Grade B
- Else : Grade C

Tasks

1. Draw the decision in simple text:
   - Condition 1: If mark ≥ 90, assign Grade A
   - Condition 2: If mark ≥ 75, assign Grade B
2. Now write Python code:

python

```
mark = int(input("Enter mark: "))
if mark >= 90:
  print("Grade A")
elif mark >= 75:
  print("Grade B")
else:
  print("Grade C")
```

3. Test with at least 3 marks and note outputs.

Arjun T
AMB 2024

Python – Intermediate Activity Handouts

Activity I1 – Problem Decomposition with Functions

Title: Breaking a Bigger Problem into Small Functions

Context
You now know variables, loops, if/else, and simple functions.
Next: learn to split a problem into clear sub-tasks.

Problem
Given a list of student marks, you must:

- Drop the lowest mark.
- Compute the new average.
- Decide pass/fail using a threshold.

Part A – Think in parts

1. Write the overall problem in one sentence:

Deciding Pass / fail based on new age

2. Identify at least three smaller tasks:
   - Task 1: Excluding the lowest mark
   - Task 2: finding new avg
   - Task 3: Decide Pass/fail
3. For each task, write a function name and its input and output:
   - Function 1 name: Deleting lowest mark
     - Input(s): list of marks
     - Output: List of marks excluding the lowest mark
   - Function 2 name: Computing the new avg
     - Input(s): List of marks and no of marks
     - Output: New avg
   - Function 3 name: Deciding Pass / fail
     - Input(s): Enter the avg and threshold
     - Output: Pass or fail

Part B – Implement functions

Fill and complete:

python

```
def drop_lowest(marks):
    # TODO: remove the smallest mark and return the new list
    return new - list

def average(marks):
    total = 0
```

```
    for m in marks:
        total = total + m
    return total / len(marks)

def is_pass(avg, threshold):
    # return True or False
    if avg >= threshold
        return True
    else:
        return False

# Main flow
marks_list = [55, 80, 90, 40]
threshold = 50

new_marks = drop_lowest(marks_list)
avg = average(new_marks)
result = is_pass(avg, threshold)
print("Marks after dropping lowest:", new_marks)
print("Average:", avg)
print("Pass?", result)
```

Reflection

- How did dividing into functions help you think more clearly?

By breaking down the Problem I's allowed me to focus on one logical step at a time and made code easy to debug and read.

Activity I3 – Reading and Writing Files

Title: Process Data from a Text File

Problem

You are given a text file marks.txt with one mark per line.
You must read it and write a summary to summary.txt.

Part A – Understand

1. Inputs (file/content): marks.txt
2. Outputs (file/content): summary.txt

Part B – Code Skeleton

python

```python
# Step 1: Read marks from file
marks = []
with open("marks.txt", "r") as f:
    for line in f:
        line = line.strip()
        if line != "":
            marks.append(int(line))
# Step 2: Compute statistics
total = sum(marks)
count = len(marks)
average = total / count
max_mark = max(marks)
min_mark = min(marks)
# Step 3: Write summary to new file
with open("summary.txt", "w") as f:
    f.write("Number of students: " + str(count) + "\n")
    f.write("Average mark: " + str(average) + "\n")
    f.write("Max mark: " + str(max_mark) + "\n")
    f.write("Min mark: " + str(min_mark) + "\n")
```

Tasks

- Add one more line to summary: count of students with mark ≥ 60.
- Write that line in the file.

Reflection

- How is processing from file different from input()?

Processing from a file store for large amt of data also it allows the data to be Permanent

Arjun T
AMB2024

Advanced Problem-Solving Lab – High-IQ Track

Activity H1 – Solve the "Hidden Problem" Behind the Stated Problem

Title: From Surface Problem to Real Problem

Goal
Train yourself to see the *real* problem, not just the stated one.

Instructions
You are given a "client statement":

"Our exam system is too slow. We want a Python script to speed it up."

1. Identify at least 3 possible *real* problems this statement might hide.
   - Hidden problem 1: Inefficient database
   - Hidden problem 2: Poorly designed Processing algorith
   - Hidden problem 3: Hardware/Network bottlenecks - Slowing down reco Processi
2. For each hidden problem, state:
   - What data would you need to confirm it?
     - Problem 1: Query execution times, database indexes data y
     - Problem 2: Printing data of exam Processing code
     - Problem 3: Server Performance metrics
3. Choose ONE hidden problem and re-frame it as a clear technical problem statement (2–3 lines):

New problem statement:

Exam system experiences high response times due to inefficient database queries that cause I/O and slow data retrieval

4. Design a very small Python experiment that could validate or reject your reframed problem (pseudo-code is fine):

python

 # Experiment idea (what will you measure, on what data, how?)

_____

Activity H3 – Self-Constrained Challenge: "Elegant Code Only"

Title: Solve with Elegance (Clarity, Minimalism, Correctness)

Goal
Produce code that is not only correct, but also minimal and beautiful.

Problem
Implement a mini "pipeline" for processing a list of tasks:

- Each task has an ID, description, and status ("todo", "doing", "done").
- You must:
  - Load tasks from a JSON file.
  - Filter by status.
  - Sort by ID.
  - Print a clean summary.

Constraints (you impose on yourself)

1. Maximum 25 lines of *real* Python (excluding blanks and comments).
2. No function longer than 7 lines.
3. Names must be meaningful and short.

Steps

1. Design your data model in 2–3 lines:

Define Class (dictionary) for task with keys id, desc and sta

2. Decide at least one thing you will *not* do to keep it minimal:

Choose to omit extensive error handling detailed logging or separate utils function for simple operation

3. Write your code here (respecting constraints):

python

 # Elegant pipeline solution

4. Reflection:
   - What did you have to sacrifice (if anything) to keep elegance?

To keep Code, elegant, I had to sacrifice explicit error a modular Printing meeting operation and Concise list Comprehen and short function ≤ 7 lines

Activity H6 – Cross-Domain Transfer

Title: Apply Known Pattern to a New Domain

Goal
Practice lifting a known idea/pattern from one domain to a different one.

Instructions
Pick a pattern you already know well, e.g.:

- Sliding window
- Two-pointer technique
- Dynamic programming table
- Producer–consumer
- Map–reduce style
- Pipeline of filters

1. Name of pattern: Sliding window
2. Original domain where you learned it (e.g., arrays, strings):
   always listings
3. New domain to apply it in (e.g., logs, time series, learning activities, medical data):
   Medical data
4. Describe how you would adapt the pattern to this new domain, in 5–8 lines (can include pseudo-code):
   Define window size, slide the window over a user's Page visitors for each window Calculate metrics identify trends or anomalies in user behavior
5. Optional: implement a small prototype showing this transfer:

python

 # Pattern transfer prototype